Quick Use Information:

Create new PlayerClassSO or EnemyClassSO, drag and drop stat CSV file into field and press PopulateFromCSV button. You can then batch modify the stats from the editor for tuning using the field StatModifiers.

The CSV Reader uses layout rules to sort the stat spreadsheet into a nested dictionary (<Class Name, <Stat Name, Stat Values>>). The SO class code is where you access the dictionary to populate the class's stats, and must be updated for added stats.

Editing and Reading the Stat Spreadsheet:

Edit/Add to the Stat Spreadsheet in editing software eg Excel, LibreOffice Calc (free). The CSV Reader is dependent on the Stat Spreadsheet layout, and the string names of classes and stats:

- String identifiers must be the exact strings referenced in code, including any whitespaces and without any extra whitespace at the end
    - Each row where you want a stat identified must have the class cell filled in with the exact class string
    - Each stat must be named with the exact stat string
- The level columns have to start at the 3rd column
- Any columns after levels can be used for calculations etc, these will be ignored provided the column header is not an int

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
1	**Classes**	**Stats**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**	**15**	**Calc Column**	**E.g. Level Up Addition Amount**
2																			
3	**Class Name**	Stat Name	Stat Value for level 1	Value for level 2	Etc.														
4																			
5																			
6	**For Example:**																		
7	**Player**	XP to level up	2000	4000	6000	8000	10000	12000	14000	16000	18000	20000	22000	24000	26000	28000	30000		2000
8	**Player**	Cumulative Experience	2000	6000	12000	20000	30000	42000	56000	72000	90000	110000	132000	156000	182000	210000	240000		
9	**Player**	Base Health	200	240	280	320	360	400	440	480	520	560	600	640	680	720	760		40
10	**Player**	Base Damage	10	15	20	25	30	35	40	45	50	55	60	65	70	75	80		5
11	**Player**	Standard Weapon bonuses	10	15	20	25	30	35	40	45	50	55	60	65	70	75	80		5
12	**Player**	Typical Ability Damage	60	90	120	150	180	210	240	270	300	330	360	390	420	450	480		
13																			
14																			
15	**Grunt**	Base Health	60	90	120	150	180	210	240	270	300	330	360	390	420	450	480		
16	**Grunt**	Base Damage	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19		
17	**Grunt**	Experience gained	80	120	160	200	240	280	320	360	400	440	480	520	560	600	640		
18																			
19	**Archer**	Base Health	42	63	84	105	126	147	168	189	210	231	252	273	294	315	336		
20	**Archer**	Base Damage	4	5	6	6	7	8	9	10	10	11	12	13	14	14	15		
21	**Archer**	Experience gained	64	96	128	160	192	224	256	288	320	352	384	416	448	480	512		
22																			
23	Each stat needs to be marked with the class name, so the CSV Reader can identify it. Stats/Rows not used will just be ignored.																		
24	The CSV Reader uses string matches to sort the data, so the class and stat names need to be string matches to the class key, and stat names on the Unity side.																		

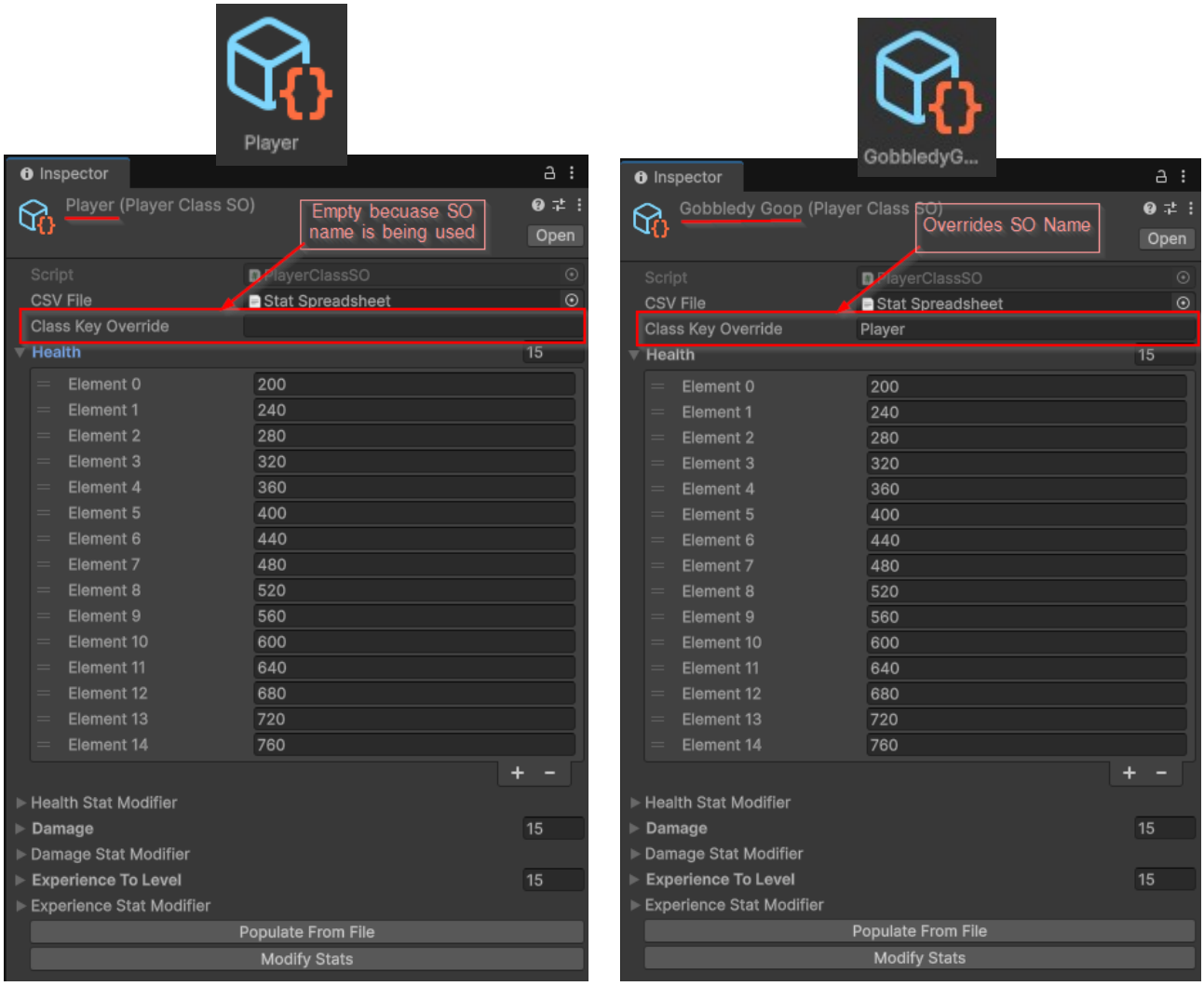
Export this spreadsheet as a .csv file and import into Unity.

The CSV Reader is a static class that sorts this data into a dictionary of classes each of which contains a dictionary of stats.

The class scriptable objects run methods when the custom editor buttons are clicked. CSVPopulableSOEditor.cs uses Unity's editor API to create these buttons.

Each SO uses it's name as the Class Key unless the Class Key Override is specified. To populate the data from the correct class section in the spreadsheet, enter the correct string in Class Key Override, or use the string as the object's name. The Class Key Override is prioritised over the object

name. E.g. Player SO doesn't need override to find player stats in spreadsheet. GobbledyGoop SO uses override string "Player" to populate from the player stats in the spreadsheet.



To help reduce string reference issues the ClassStatConsts class holds const string references which are used via variable name.

```
if (data.ContainsKey(ClassStatConsts.DAMAGE))
{
    damage = data[ClassStatConsts.DAMAGE];
}
```

```
8 references
public class ClassStatConsts
{
    //Character
    2 references
    public const string HEALTH = "Base Health";
    2 references
    public const string DAMAGE = "Base Damage";

    2 references
    public class Player
    {
        2 references
        public const string EXP_TO_LEVEL = "Experience To Level";
    }

    2 references
    public class Enemies
    {
        2 references
        public const string EXP_FROM_DEATH = "Experience From Death";
    }
}
```

How to add your own classes or stats:

Follow the spreadsheet layout rules to add classes and stats and then export spreadsheet as .csv file and import csv file into Unity.

To populate a new class create a new PlayerClassSO or EnemyClassSO and use the exact class string from the csv as either the SO name or as the Class Key Override.

To populate new stats: If the stat/s you added are shared between player and enemy classes, add the following to CharacterClassSO.cs, otherwise add to either PlayerClassSO.cs or EnemyClassSO.cs:

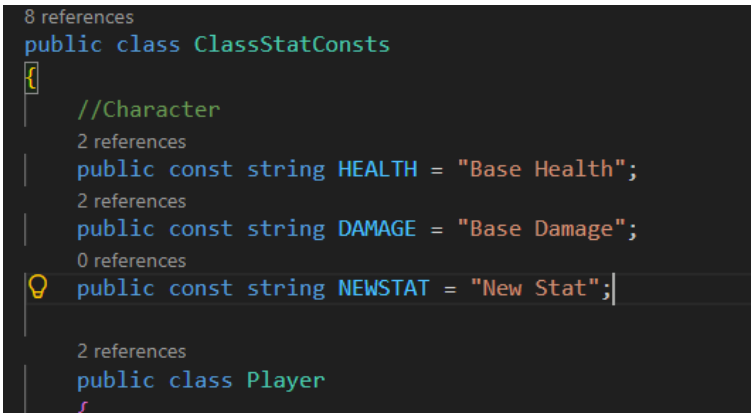
Add to the top of the class the following to create stat array and modifier fields:

```
0 references | Unity Serialized Field
[SerializeField] float[] newStat;
0 references | Unity Serialized Field
[SerializeField] StatModifer newStatModifier;
```

Also add an accessor:

```
0 references
public float GetNewStat(int level) //stat accessor
{
    if (newStat.Length < level) return newStat[newStat.Length - 1];
    return newStat[level - 1];
}
```

Go to ClassStatConsts.cs and add a new constant for any stat types you created putting it in the appropriate subclass where applicable:



Back in the CharacterClassSO.cs (or children), you have to add three more things:

1. In the PopulateData() method add the following to populate newStat with data from the CSVReader dictionary:

```
if (data.ContainsKey(ClassStatConsts.NEWSTAT))
{
    newStat = data[ClassStatConsts.NEWSTAT];
}
```

2. In the ModifyData() method add this:

```
if (newStatModifier.modifierMethod != ModifierMethod.None)
{
    ModifyField(newStat, newStatModifier);
}
```

3. And finally add the following to the ResetModifiers() method:

```
newStatModifier = default;
```